工学硕士学位论文

面向通用通信处理器的算子复用调度、跨模态知识图谱构建与 GraphRAG 智能纠错研究

学生姓名: 叶艺烽
学　号: 2023E8013282135
学科专业: 计算机技术
研究方向: 通信处理器
指导教师: 莫志锋（高级工程师）
培养单位: 中国科学院计算技术研究所

中国科学院大学
2026 年

摘　要

5G 商用与 6G 研发持续推进，LTE、5G NR、Wi-Fi 6/7、NTN 等多制式长期共存，导致通信终端与基站基带处理器需要在不同协议链路之间快速切换与复用。传统“一个协议一套硬件”的烟囱式架构在多模时代面临面积墙、功耗墙与灵活性墙：多套专用 IP 的线性堆叠引入大量闲置逻辑，晶体管利用率下降；协议跨代演进带来不可持续的工程迭代成本；同时 6G AI-Native 愿景要求物理层具备可解释、可执行的自愈闭环能力，传统规则库与黑盒模型难以在毫秒级异常中完成根因定位与安全处置。

围绕上述问题，本文提出“算子复用调度—跨模态知识图谱—GraphRAG 智能纠错”三位一体体系：在硬件侧基于 OFDM/MIMO 物理层算法的数学同构性提取符号级通用算子，并设计支持影子寄存器与原子更新的多模动态重构机制；在系统侧将带重配置代价的异构资源受限 DAG 调度建模为多目标优化问题，构建 HEFT、DLS、遗传算法与知识图谱引导遗传算法等调度器，并以 makespan、切换次数、有效重配置惩罚、等待/响应时间、资源利用率与吞吐等指标进行对比评测；在运维侧构建覆盖协议规范、RTL 实现与运行日志的跨模态通信知识图谱，基于“向量检索 + 子图遍历”的混合召回策略与结构化提示词工程，驱动 GraphRAG 生成可审计的寄存器级修复动作，并通过白名单过滤与危险动作拦截保障执行安全。

基于软硬协同仿真验证，本文方案在典型多模调度场景中显著降低协议切换开销并提升流水线利用率；在同步参数失配与信噪比突变等故障场景中，GraphRAG 能够在图谱证据约束下生成可执行修复策略并完成闭环验证，较传统文档检索式 RAG 与规则库具有更高的鲁棒性与可解释性。本文研究为面向 6G 的通用通信处理器架构演进与物理层自治运维提供了可落地的方法与工程参考。

关键词：通用通信处理器；符号级算子复用；带切换代价的 DAG 调度；跨模态知识图谱；GraphRAG；智能纠错

Abstract

The coexistence of multiple wireless standards (e.g., LTE, 5G NR, Wi-Fi 6/7, and NTN) forces modern baseband processors to switch among heterogeneous protocol pipelines with stringent latency and energy constraints. Conventional siloed designs that stack per-standard hardware IP blocks suffer from area and power walls, poor silicon utilization, and limited evolvability. Meanwhile, the 6G AI-native vision calls for explainable and actionable self-healing capabilities at the physical layer, where rule-based systems and black-box models often fail to produce safe, executable remedies under millisecond-scale anomalies.

This thesis proposes an integrated framework that couples (1) symbol-level operator reuse with fast mode switching, (2) multi-objective scheduling under reconfiguration overhead, and (3) cross-modal knowledge graphs with GraphRAG-based fault diagnosis and correction. We first extract common operators from OFDM/MIMO signal processing based on mathematical isomorphism and implement a reconfigurable architecture with shadow registers and atomic updates. We then formulate resource-constrained DAG scheduling with protocol switching penalties and implement multiple schedulers (HEFT, DLS, genetic algorithms, and knowledge-guided genetic algorithms), evaluated by makespan, protocol switches, effective reconfiguration penalty, waiting/response time, utilization, and throughput. Finally, we construct a cross-modal communication knowledge graph that links standards, RTL implementation, and runtime logs, and design a GraphRAG pipeline with hybrid retrieval (vector search plus subgraph traversal), structured prompting, and safety filters to generate register-level corrective actions with verifiable evidence.

Experimental results in a soft/hardware co-simulation environment show that the proposed approach reduces protocol switching overhead while improving pipeline utilization, and enables robust, explainable, and executable recovery for representative fault scenarios such as synchronization parameter mismatch and SNR drops. The framework provides practical insights toward evolvable multi-standard baseband processors and physical-layer autonomy in future 6G systems.
Keywords: multi-standard baseband; operator reuse; DAG scheduling with switching cost; cross-modal knowledge graph; GraphRAG; fault diagnosis and correction

目录

摘要 **i**

Abstract **ii**

符号与缩略语 **vii**

第一章 绪论 **1**
1.1 研究背景 1
1.2 研究问题与挑战 1
1.2.1 算子复用下的调度与切换开销 1
1.2.2 跨源知识割裂与语义鸿沟 1
1.2.3 物理层纠错的可执行性与安全性 1
1.3 研究目标与总体思路 2
1.4 本文主要工作与创新点 2
1.5 论文结构安排 2

第二章 技术基础与系统框架 **3**
2.1 多制式基带处理与符号级算子抽象 3
2.2 多模接收机原型与动态重构机制 3
2.2.1 顶层结构与算子复用 3
2.2.2 影子寄存器与原子更新 3
2.2.3 模式控制与流控 3
2.2.4 关键算子实现要点 4
2.3 带重配置代价的资源受限 DAG 调度模型 4
2.4 带重配置代价的资源受限 DAG 调度模型 4
2.5 跨模态通信知识图谱基础 4
2.6 向量检索与语义编码模型 4
2.7 系统总体架构 5

第三章 面向算子复用的调度优化与评测 **6**
3.1 问题建模 6
3.2 调度器设计与实现 6
3.2.1 HEFT 基线 6
3.2.2 DLS 基线 6
3.2.3 遗传算法与知识引导遗传算法 6
3.2.4 强化学习探索性基线 6

3.3 实验平台与复现性 7
3.4 评测指标体系与实验设置 7
3.5 实验结果与分析 7
3.6 小结 7

第四章 跨模态通信知识图谱构建与质量评估 8
4.1 设计目标与总体流程 8
4.2 领域本体与关系建模 8
4.3 三源知识抽取 8
4.3.1 协议文档知识抽取 8
4.3.2 RTL/寄存器实现知识抽取 8
4.3.3 运行日志与故障表征抽取 8
4.4 跨模态融合与实体对齐 9
4.5 质量评估与可视化 9
4.6 与调度与纠错的联动 9
4.7 小结 9

第五章 基于 GraphRAG 的物理层智能纠错机制 10
5.1 问题定义与总体架构 10
5.2 混合检索：向量召回与子图遍历 10
5.2.1 查询构造与语义向量化 10
5.2.2 以故障为锚点的图拓扑遍历 10
5.3 结构化提示词工程与决策生成 10
5.4 动作安全过滤与可执行闭环 10
5.5 结构化动作表示与硬件接口 11
5.6 实验设计与结果分析 11
5.7 小结 11

第六章 总结与展望 12
6.1 工作总结 12
6.2 不足与讨论 12
6.3 未来工作展望 12

附录：工程实现与实验复现说明 14
.1 代码仓库结构 14
.2 调度实验复现 14
.3 知识图谱构建与可视化 14
.4 GraphRAG 纠错仿真 14

插图

表格

符号与缩略语

符号/缩写	说明
OFDM	Orthogonal Frequency Division Multiplexing，正交频分复用
MIMO	Multiple-Input Multiple-Output，多输入多输出
LTE	Long Term Evolution，4G 演进系统
NR	New Radio，5G 空口
DAG	Directed Acyclic Graph，有向无环图
RAG	Retrieval-Augmented Generation，检索增强生成
GraphRAG	Graph-based RAG，基于图检索增强生成
AXI4-Stream	AMBA AXI4 数据流接口
AXI4-Lite	AMBA AXI4 轻量控制总线
CSR	Control and Status Register，控制状态寄存器
SNR	Signal-to-Noise Ratio，信噪比
SCS	Subcarrier Spacing，子载波间隔

第一章　绪论

1.1　研究背景

5G 商用与 6G 研发持续推进，多制式长期共存已成为现实网络的常态。LTE 面向广域覆盖，5G NR 面向高容量与低时延场景，Wi-Fi 6/7 面向室内高吞吐接入，NTN 拓展空天地一体化覆盖边界。不同协议在子载波间隔、循环前缀、同步序列、信道编码与多天线处理等方面差异显著，使得基带处理器既要支持跨制式快速切换，也要在有限芯片面积与功耗预算下保持高吞吐与稳定性。

传统“一个协议一套硬件”的烟囱式基带架构具有工程可控的优势，但在多模时代暴露出三类结构性瓶颈：其一，面积线性增长导致暗硅现象与布线/时钟压力；其二，多套专用核的冗余带来动态与静态功耗上升；其三，协议跨代演进导致难以通过软件升级平滑适配，工程迭代成本高。面向 6G AI-Native 网络，物理层还需要具备异常感知、根因定位与安全处置能力，从而实现更高等级的自治运维。现有规则库方案覆盖有限，传统文档切片式 RAG 缺乏结构化推理并可能产生不可执行的“幻觉”动作，黑盒深度学习模型难以给出可解释的修复指令。

1.2　研究问题与挑战

1.2.1　算子复用下的调度与切换开销

多制式复用同一套硬件算子能够显著节省面积与功耗，但带来协议上下文频繁切换、流水线冲刷与状态收敛等额外开销。面向符号级算子复用的数据通路，需要在满足任务依赖与资源约束的前提下，优化任务映射与执行顺序，并显式建模重配置代价，避免出现“乒乓切换”导致的吞吐骤降。

1.2.2　跨源知识割裂与语义鸿沟

通信链路知识分散于三类异构数据源：协议标准文档（非结构化文本）、RTL/寄存器映射（半结构化实现细节）、运行日志（非结构化症状）。传统检索与人工经验难以建立“故障现象—模块—寄存器—修复动作”的因果链路，导致定位慢、不可扩展且难以复用。

1.2.3　物理层纠错的可执行性与安全性

物理层控制的动作往往落到寄存器级写入与模块级复位，任何错误动作都可能引发更严重的系统风险。因此，纠错机制不仅要“能推理”，还要保证动作在硬件约束与协议约束下可执行、可审计，并具备对危险动作的拦截能力。

1.3　研究目标与总体思路

本文以“算子复用调度—跨模态知识图谱—GraphRAG 智能纠错”为主线，构建从体系结构到运维闭环的统一方案。

1. 在硬件侧，抽象符号级通用算子并实现运行时可配置的多模接收机原型，采用影子寄存器与原子更新支撑微秒级模式切换。
2. 在系统侧，建立带重配置代价的资源受限 DAG 调度模型，实现多种调度器并建立多维指标体系进行对比评测。
3. 在运维侧，构建连接协议、实现与运行的跨模态知识图谱，设计 GraphRAG 推理流水线，将故障上下文映射为可执行修复动作并闭环验证。

1.4　本文主要工作与创新点

本文的主要工作与创新点概括如下。

1. 提出面向多制式 OFDM/MIMO 链路的符号级通用算子抽象方法，并给出可配置的 RTL 实现与动态重构机制。
2. 建立带切换代价的异构资源受限 DAG 调度评测框架，给出知识图谱引导的遗传调度方法，并用多维指标刻画其在“高切换代价陷阱场景”下的优势。
3. 构建跨模态通信知识图谱，提出“向量检索 + 子图遍历”的混合召回与结构化提示词工程，并通过白名单与危险动作过滤实现可执行的 GraphRAG 纠错闭环。

1.5　论文结构安排

本文共分六章。第一章介绍研究背景、问题与本文贡献；第二章给出关键技术基础与系统总体框架；第三章阐述算子复用调度建模、算法实现与实验评测；第四章阐述跨模态知识图谱构建、融合与质量评估；第五章阐述 GraphRAG 智能纠错机制与闭环验证；第六章总结全文并给出后续展望。

第二章　技术基础与系统框架

2.1　多制式基带处理与符号级算子抽象

尽管 LTE、5G NR 与 Wi-Fi 在帧结构、子载波间隔与导频设计上存在差异，其物理层接收链路仍遵循“同步—变换—估计—检测”的共同数学范式。以 OFDM 解调为例，接收端的核心计算可归结为对离散时域信号进行 FFT 变换，不同协议的差异主要体现在 FFT 点数、循环前缀与子载波映射规则等参数。类似地，滑动窗口相关、能量累计与星座判决等操作在多协议间具有数学同构性，因此可进一步抽象为参数可配置的符号级算子。

本文将符号级算子定义为：在物理层数据流中能够独立完成特定数学变换、并具有标准化输入输出接口的最小功能单元。每个算子由输入信号流、配置参数向量与输出信号流描述。面向多模复用与快速切换目标，本文优先选择同步检测与解调映射等具有高计算密集度与强同构性的数据通路环节作为核心算子实现对象，并在后续章节给出可配置 RTL 设计与验证。

2.2　多模接收机原型与动态重构机制

2.2.1　顶层结构与算子复用

为验证符号级算子复用的工程可行性，本文实现一个多模接收机 RTL 原型，支持 LTE、NR 与 Wi-Fi 三种工作模式切换。顶层模块将同步检测与解调映射算子在时间片上复用，并提供多天线并行实例化以覆盖典型 MIMO 场景。模式切换由控制模块驱动，向各算子下发配置参数并触发原子更新。

2.2.2　影子寄存器与原子更新

多模切换的核心难点在于避免配置中间态造成的流水线冒险与瞬态错误。本文采用影子寄存器机制：配置参数先写入 shadow，待满足排空与保护时序后，通过更新触发信号一次性提交到 active 配置，从而实现原子生效。该机制与后续 GraphRAG 的“写 shadow + trigger update”的纠错动作形式保持一致，便于形成统一的软硬协同控制接口。

2.2.3　模式控制与流控

为保证切换过程中数据一致性，控制模块在检测到模式变更请求后先进入 drain 状态阻断输入，等待流水线排空，再进入配置与 warmup 状态，最后恢复正常运行。数据平面采用标准握手机制支持反压，控制平面通过全局 ready 信号配合完成切换过程的节拍管理。

2.2.4 关键算子实现要点

同步检测算子以滑动窗口相关为核心，采用循环缓冲区保存历史采样点，并通过递归滑动累加器在每个周期以常数复杂度更新能量统计量，避免窗口长度增长导致的加法器树爆炸。在多模场景下，窗口长度与延迟长度可由 FFT 点数等参数派生，并在 config_update 触发时完成状态清零以避免跨模式混叠能量造成的误检。

解调映射算子将复数星座点映射为比特序列，并通过可配置调制类型选择判决逻辑。该算子面向复用与快速切换，保持接口与控制参数的统一化，使其能够在 LTE/NR/Wi-Fi 的不同调制阶数配置下复用同一套数据通路。

2.3 带重配置代价的资源受限 DAG 调度模型

2.4 带重配置代价的资源受限 DAG 调度模型

将物理层链路抽象为有向无环图（DAG），节点表示算子任务，边表示数据依赖。系统包含若干类异构算子资源，每类资源支持多种协议上下文；当相邻任务协议上下文不一致时，需要付出重配置/模式切换代价。本文关注的优化目标不仅是最小化总完工时间（makespan），还包括降低切换次数与切换带来的有效延迟，同时提升资源利用率并降低等待时间。

为便于论文实验可复现与与实现保持一致，本文采用以下多维指标作为评测基准：

- makespan：任务图完成时间；
- protocol switches：各资源上相邻任务协议不同的切换次数；
- effective reconfiguration penalty：从调度日志反推的切换导致的有效延迟；
- waiting/response：任务从 ready 到启动的等待时间与到完成的响应时间；
- utilization：资源忙闲比与负载均衡程度；
- throughput：单位时间完成任务数。

2.5 跨模态通信知识图谱基础

知识图谱以实体—关系的方式对领域知识进行结构化建模。本文围绕通信链路控制与运维需求设计领域本体，核心实体包括 Protocol、Module、Register、Error 与 Solution，核心关系包括 implements、configures、connects_to、causes 与 fixes。通过将协议标准、RTL 实现与运行日志统一映射至同一语义空间，可支持多跳因果链查询，并为上层推理与动作生成提供证据约束与安全校验依据。

2.6 向量检索与语义编码模型

为解决自然语言描述与图谱实体之间的语义匹配问题，本文引入句向量编码模型，将故障描述、日志文本与图谱节点文本编码为稠密向量，在向量空间内通过余弦相似度实现近邻检索。在工程实现中，本文采用基于 SentenceTransformer 的指令式编码方式，并实现轻量级内存索引以支持快速检索与可复现评测。

2.7　系统总体架构

本文系统遵循“数据平面—控制平面—智能运维”的三层协同架构。数据平面由可复用算子与流水线互连组成，通过影子寄存器与原子更新机制实现多模切换；控制平面负责任务编排与调度策略执行，显式建模切换开销并通过评测指标体系进行优化；智能运维层构建跨模态知识图谱，并以 GraphRAG 推理实现从故障感知到寄存器级修复动作的闭环。

第三章　面向算子复用的调度优化与评测

3.1　问题建模

算子复用场景下的调度问题可建模为带重配置代价的异构资源受限 DAG 调度。设任务集合为 $T = \{t_i\}$，依赖边集合为 E，每个任务具有算子类型、执行时长与协议属性。硬件资源集合为 $R = \{r_k\}$，每个资源在任意时刻最多执行一个任务；当资源上相邻任务协议不一致时，引入重配置代价 δ。目标是在满足依赖与资源约束的前提下，优化 makespan，并兼顾切换开销与资源利用率等指标。

3.2　调度器设计与实现

3.2.1　HEFT 基线

HEFT 通过 upward-rank 计算静态优先级列表，并结合最早完成时间进行资源分配。在本项目中 HEFT 作为典型的静态列表调度基线，与其他算法共享同一仿真环境与指标计算口径。

3.2.2　DLS 基线

DLS 采用动态列表调度，综合静态层级与最早可开始时间决策下一任务。为刻画多模切换的代价，本文在仿真环境中将重配置惩罚作为硬件参数统一管理，使不同算法在相同 penalty 设定下可公平对比。

3.2.3　遗传算法与知识引导遗传算法

遗传算法在任务优先级序列空间进行搜索，个体编码为拓扑合法的任务排列，适应度以 makespan 与切换开销的加权组合表示。知识引导遗传算法在初始化、交叉与变异阶段引入协议聚类与上下文复用偏置，以降低频繁切换的概率，并提升在高 penalty 场景下的收敛速度与解质量。

3.2.4　强化学习探索性基线

为验证“基于策略学习的调度”在该问题上的可行性，本文实现 step-wise 调度环境：每一步从 ready set 中选择下一任务，调度器使用同一 dispatch 规则执行并推进环境状态。该实现用于与启发式与进化算法进行补充性对比，并为后续引入图神经网络编码提供接口基础。

3.3　实验平台与复现性

本文调度实验基于统一的仿真环境完成：输入为任务 DAG（节点包含算子类型、执行时长与协议属性），输出为调度日志（每个任务的资源分配、开始/结束时间与协议切换信息）。所有算法的评测指标由同一指标计算模块生成，从而避免口径不一致导致的对比偏差。实验脚本支持单用例对比与批量 benchmark，可复现实验结果并便于统计多组工作负载下的均值与分位数表现。

3.4　评测指标体系与实验设置

本文采用多维指标体系对调度器进行系统评估，包括 makespan、切换次数、有效切换惩罚、等待/响应时间、平均利用率与负载均衡程度等。实验用例包含高切换代价的“陷阱场景”与多链竞争场景，用于体现不同算法在复杂多模工作负载下的稳定性与优势。

3.5　实验结果与分析

本节建议按“单用例对比—批量统计—消融实验”组织结果呈现：

1. 单用例：展示 HEFT/DLS/GA/KG-GA 的多指标对比表与关键指标图；
2. 批量统计：对多组随机用例输出箱线图或均值方差统计；
3. 消融：关闭知识引导、改变重配置惩罚、改变资源数量等，分析方法有效边界。

3.6　小结

本章围绕算子复用场景下的调度优化，给出问题建模、调度器实现与多维指标评测方法。实验表明，在高切换代价与多模交错任务场景下，引入知识引导的全局搜索能有效减少无意义的上下文切换并提升整体执行效率，为后续将知识图谱与物理层自愈闭环联动奠定基础。

第四章　跨模态通信知识图谱构建与质量评估

4.1　设计目标与总体流程

为解决通信链路知识在协议文档、RTL 实现与运行日志之间的割裂问题，本章构建跨模态通信知识图谱，将“标准—实现—运行”三层知识统一到可查询、可推理的语义空间。总体流程包括：本体建模、三源知识抽取、跨模态融合与实体对齐、图谱质量评估与可视化。

4.2　领域本体与关系建模

本文以 Protocol、Module、Register、Error、Solution 为核心实体集合，建立 implements、configures、connects_to、causes、fixes 等关系集合，并为实体设计协议参数、寄存器地址/取值域、模块拓扑与资源消耗等属性。该本体使后续能够沿“错误现象—模块—寄存器—协议约束—修复方案”形成可追溯的因果链路。

4.3　三源知识抽取

4.3.1　协议文档知识抽取

面向 3GPP/IEEE 等标准文档，抽取协议参数域、约束关系与枚举定义。该过程需解决文档结构复杂、参数归一化与歧义消解等问题。工程实现可采用规则与大模型辅助抽取相结合的方式，并将结果映射为 Protocol 实体及其属性与约束关系。

4.3.2　RTL/寄存器实现知识抽取

面向 Verilog RTL，抽取模块层级、实例化关系、端口连接、配置相关信号与寄存器映射，从而建立 Module 与 Register 实体，并构建 connects_to 与 configures 关系。该过程为后续纠错动作的合法性校验提供“地址白名单与约束源”。

4.3.3　运行日志与故障表征抽取

面向运行日志与监测数据，将非结构化症状归一为 Error 实体，并补充关键运行上下文(SNR、SCS、模式、时间戳等)。在工程实现中可结合日志模板挖掘与语义匹配，实现对新故障描述的聚类与关联。

4.4　跨模态融合与实体对齐

跨模态融合的核心是识别不同数据源中同一实体的对应关系，并合并或链接其属性与关系。本文采用名称规范化、结构上下文与嵌入相似度结合的方法进行实体对齐；对冲突关系引入置信度或一致性检查策略，以降低语义漂移风险。该阶段输出的图谱既保留协议意图，又具备硬件落地细节与运行时观测证据。

4.5　质量评估与可视化

为支持论文实验与系统迭代，本章引入图谱规模、连通性、密度、重复度与一致性等指标进行质量评估，并提供可视化输出以便审阅与调试。评估与可视化结果可用于后续的消融实验，例如对比“启用/禁用实体对齐”的图谱质量变化及其对 GraphRAG 纠错性能的影响。

4.6　与调度与纠错的联动

本图谱在本文体系中承担双重角色。一方面，图谱可为调度算法提供“算子—协议兼容性”“模式切换代价”等约束信息，降低无效解比例并缓解高 penalty 场景下的搜索难度；另一方面，图谱为纠错提供“故障—模块—寄存器—协议约束—解决方案”的证据链，支持将自然语言故障描述映射为可执行的寄存器级动作，并在动作生成后对地址与取值域进行合法性校验。

4.7　小结

本章构建了跨模态通信知识图谱，并给出了本体设计、三源抽取、融合对齐与质量评估方法。该图谱为调度优化提供约束信息，也为后续 GraphRAG 推理提供结构化证据与安全校验依据。

第五章　基于 GraphRAG 的物理层智能纠错机制

5.1　问题定义与总体架构

物理层故障通常表现为同步丢失、解调失败、CRC 异常、缓冲区溢出或模式切换超时等症状，其根因可能来自信道条件突变、参数配置失配、硬件状态不一致或资源调度引发的时序冲突。纠错系统需要在短时间内完成故障定位与动作下发，同时保证动作安全、可执行、可审计。

本文采用快慢双环架构：L1 快环由硬件规则与状态机实现，负责微秒级异常检测与基础保护；L2 慢环由 GraphRAG Agent 实现，接收 L1 上报的错误码与上下文，进行混合检索、多跳推理与结构化动作生成，并通过影子寄存器与触发机制完成原子更新与闭环验证。

5.2　混合检索：向量召回与子图遍历

5.2.1　查询构造与语义向量化

系统将硬件错误码与上下文（协议、SNR、SCS、模式选择、重配置延迟等）转化为自然语言查询文本，并使用句向量模型编码为查询向量，在向量索引中召回语义相近的历史故障、协议约束节点或解决方案节点。

5.2.2　以故障为锚点的图拓扑遍历

在图结构中，以 Error 或相关 Module 节点为锚点，通过 predecessors/successors 扩展实现多跳遍历，召回与故障因果相关的 Module、Register 与 Protocol 节点，构建上下文子图。最终将向量召回与图遍历结果取并集，得到兼顾语义泛化与物理因果一致性的上下文集合。

5.3　结构化提示词工程与决策生成

检索得到的子图以三元组或模板化自然语言进行序列化，与运行态上下文共同构成大模型提示词。为支持多轮尝试与自反思，系统将历史动作与结果以结构化形式加入提示中，促使模型避免重复无效动作并逐步收敛至可行修复策略。

5.4　动作安全过滤与可执行闭环

物理层动作必须满足地址白名单、参数取值域与危险动作拦截策略。本文实现规则过滤器，对不存在的寄存器地址、越界参数值与高风险全局复位等动作进行拦截，保证输出动作可执行

且安全。动作通过写入影子参数区并触发更新完成原子生效，随后通过物理层健康检查验证修复是否成功；若失败则进入下一轮推理，直至成功或达到最大重试次数。

5.5　结构化动作表示与硬件接口

为降低大模型输出的歧义性并便于执行校验，本文将纠错动作约束为结构化序列，每条动作由寄存器地址、字段名、目标值与生效方式组成。典型动作包括写入影子配置寄存器、触发更新、清除统计计数器与进入保护模式等。硬件侧通过控制总线提供 CSR 访问接口，并为关键控制动作提供单独的触发位，保证动作能够以“写 shadow + trigger update”的形式原子生效。

5.6　实验设计与结果分析

本节建议按“基线对比—典型场景—极端场景失败保护—消融实验”组织：

1. 基线对比：传统关键字检索式 RAG 与 GraphRAG 的检索耗时与动作质量对比；
2. 典型场景：同步参数失配、SNR 突变导致的 CRC 异常；
3. 极端场景：信道深衰落导致物理不可恢复时，系统应安全退出并提示人工介入；
4. 消融：关闭语义检索、关闭图遍历、关闭过滤器，观察成功率与风险变化。

5.7　小结

本章给出面向物理层的 GraphRAG 智能纠错机制，通过混合检索、结构化提示与安全过滤，将大模型生成约束在图谱证据与硬件合法性范围内，实现可解释、可执行的寄存器级自愈闭环。

第六章　总结与展望

6.1　工作总结

本文围绕多制式共存下通用通信处理器的体系结构复用与物理层自治运维需求，提出“算子复用调度—跨模态知识图谱—GraphRAG 智能纠错”三位一体方案，并完成了软硬协同的原型实现与评测验证。

1. 在硬件侧，基于 OFDM/MIMO 物理层算法同构性抽象符号级通用算子，设计支持运行时参数配置的多模接收机原型，并采用影子寄存器与原子更新机制支撑多模切换。
2. 在系统侧，建立带重配置代价的资源受限 DAG 调度模型，实现 HEFT、DLS、遗传算法与知识引导遗传算法等调度器，并构建多维指标体系对多模工作负载进行对比评测。
3. 在运维侧，构建覆盖协议规范、RTL 实现与运行日志的跨模态通信知识图谱，提出混合检索与结构化提示词工程，结合安全过滤与闭环验证，实现可解释、可执行的 GraphRAG 纠错流程。

6.2　不足与讨论

尽管本文已完成系统性原型与仿真验证，仍存在若干不足：

1. 图谱覆盖度与一致性仍依赖抽取质量，极端硬件故障与异常链路退化的知识仍需补充；
2. 纠错闭环目前以仿真接口验证为主，真实板级环境下的时序、并发与噪声因素需要进一步验证；
3. 大模型推理仍存在不确定性，需引入更强的结构化约束、工具调用验证与在线策略更新机制。

6.3　未来工作展望

面向后续研究与工程落地，本文计划从以下方向进一步拓展：

1. 图谱工程化：引入图数据库与向量索引服务，实现增量更新、冲突消解与在线质量监控；
2. 纠错可信化：引入形式化约束与可验证执行计划，将寄存器写入与模式切换动作纳入可证明安全的策略空间；
3. 联动优化：将调度异常与硬件故障诊断联动，形成“调度—配置—纠错”统一策略闭环；
4. 学习型调度：引入图神经网络与强化学习，在图约束下学习面向多目标的调度策略并与知识图谱融合。

参考文献

附录：工程实现与实验复现说明

.1 代码仓库结构

本文实验与原型实现对应的项目根目录为 `/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`。其中：

- 调度评测框架：`/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`/ai_scheduler
- 知识图谱构建：`/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`/knowledge_graph_construct
- GraphRAG 纠错闭环：`/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`/rag_error_correction
- 多模接收机 RTL：`/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`/multi_standard_rx_proj

.2 调度实验复现

建议在仓库根目录执行：

```
python ai_scheduler/main.py
python ai_scheduler/benchmark.py --scenario greedy_trap --num 20 --seed 1
```

.3 知识图谱构建与可视化

建议在仓库根目录执行：

```
python knowledge_graph_construction/main_kg.py
```

生成的图谱与可视化文件位于 `/mlx_devbox/users/yeyifeng.0101/playground/Ope_Item`/knowledge_graph_

.4 GraphRAG 纠错仿真

建议在仓库根目录执行：

```
python rag_error_correction/main_rag.py
```